

MDTS Device Sensor + PyQt5 Runtime

TL;DR

`device_sensor_pyqt5` 는 MDTS의 현장 장비 모듈이다. Raspberry Pi는 센서 서버와 로컬 저장소 역할을 맡고, Jetson Nano는 PyQt5 기반 의료 모니터링 화면과 외상 촬영 UI를 실행한다. 웹 대시보드는 이 모듈을 통해 실제 센서 값, 현재 선택 선원, 집중 관리 상태, 외상 카메라 프레임, 외상 분석 결과를 받아온다.

이 모듈의 목표는 선박 현장에서 센서 측정, 환자 선택, 외상 촬영, 응급처치 가이드 표시를 장비 화면에서 직접 수행하고, 동일 상태를 웹 대시보드와 동기화하는 것이다.

시스템 역할

장비	역할
Raspberry Pi	센서 수집, Flask API, 로컬 MariaDB 저장, USB CSV 백업, 집중 관리 상태 저장, Jetson 제어 프록시
Jetson Nano	PyQt5 GUI, 카메라 촬영 UI, 외상 분석 화면, 응급처치 가이드, 웹 제어 API
Windows/Web Dashboard	운영자 대시보드, AI 질의응답, 환자 차트, 원격 촬영 제어
Campus MySQL	선원 정보와 원격 바이탈 공유 DB

네트워크 구성

기본 시연 환경은 동일 공유기 네트워크와 Jetson-Pi 직접 LAN 연결을 함께 고려한다.



오프라인 또는 직접 LAN 연결 시에는 Pi와 Jetson의 고정 IP를 기준으로 통신한다. 실제 운영 전에는 네트워크 모드별 IP와 route 우선순위를 고정해야 한다.

디렉터리 구조

```
device_sensor_pyqt5/
├─ main.py                # Jetson Nano PyQt5 GUI 및 제어 API
├─ sensor_server_rpi.py   # Raspberry Pi Flask 센서 서버
├─ sensor_handler.py      # 센서 처리 참고 모듈
├─ diag_sensor.py         # 센서 진단 유틸리티
├─ transfer.py            # SFTP 전송 유틸리티
├─ deploy_jetson.py       # Jetson 배포 스크립트
├─ deploy_rpi.py          # Raspberry Pi 배포 스크립트
├─ deploy_all.py          # 통합 배포 스크립트
├─ requirements.txt
├─ run_mdts.sh
├─ start.bat
└─ CPR/, assets files
```

주요 파일 설명

sensor_server_rpi.py

Raspberry Pi에서 실행되는 Flask 서버다.

담당 기능은 다음과 같다.

- MAX30102 심박/산소포화도 수집
- MLX90614 체온 수집
- ADS1115 기반 혈압/호흡 데이터 수집
- 로컬 MariaDB `vital_logs` 저장
- 원격 `tb_vital` 저장
- USB CSV 백업
- 센서 ON/OFF와 DB 저장 게이트 제어
- 웹/PyQt5 집중 관리 선원 상태 공유
- Jetson PyQt5 카메라/외상 분석 제어 프록시

main.py

Jetson Nano에서 실행되는 PyQt5 GUI다.

담당 기능은 다음과 같다.

- 모니터링 화면
- 선원 통합 관리 화면
- 센서 ON/OFF 화면

- 외상 카메라 촬영 화면
- 외상 AI 분석 결과 화면
- 외상 응급처치 가이드 화면
- 웹 대시보드 제어용 HTTP API :5055

데이터 흐름

1. 실시간 바이탈 흐름

Sensor Hardware

- Raspberry Pi sensor_server_rpi.py
- /vitals API
- Jetson PyQt5 SensorDataFetcher
- PyQt5 MonitorScreen

Sensor Hardware

- Raspberry Pi local MariaDB vital_logs
- USB CSV backup
- Campus MySQL tb_vital
- Web Dashboard / AI Backend

2. 현재 선택 선원 흐름

PyQt5 CrewScreen select patient

- Pi /crew POST
- Web App polling /api/sensor/crew
- Web activePatient 변경

웹에서 임시로 다른 선원을 조회하는 것은 가능하지만 PyQt5의 선택 선원을 바꾸지 않는다. 실제 센서 저장 기준은 PyQt5에서 선택한 선원이다.

3. 집중 관리 상태 흐름

Web CrewManagement + 환자로 전환

- Node API /api/sensor/crew/focus
- Pi /crew/focus
- /home/pi/mdts_focused_crew_state.json
- PyQt5 CrewScreen polling

PyQt5 CrewScreen 환자전환/등록

- Pi /crew/focus
- Web App polling
- localStorage mdts_crew_list 갱신

이 구조는 원격 DB 스키마를 변경하지 않는다. 시연용으로 웹과 PyQt5의 상태를 빠르게 맞추기 위한 별도 공유 상태다.

4. 외상 촬영 흐름

```
Web trauma button
→ Node /api/trauma/pyqt5/start
→ Pi /trauma/start
→ Jetson /trauma/start
→ PyQt5 TraumaScanScreen
→ Web frame polling
→ PyQt5 analysis result
→ Web Emergency page
→ PyQt5 TraumaGuideScreen
```

웹에서 데이터 전송, 처치 종료, 진단 모드 종료, 취소 를 누르면 `/trauma/stop` 경로로 PyQt5 카메라 송출과 분석 타이머를 중단한다.

Raspberry Pi Sensor API

Method	Endpoint	역할
GET	<code>/vitals</code>	최신 센서값 조회
GET	<code>/manual</code>	수기 입력 상태 조회
POST	<code>/manual</code>	수기 입력값 저장
POST	<code>/manual/clear</code>	수기 입력값 초기화
GET	<code>/recording</code>	DB 저장 게이트 상태 조회
POST	<code>/recording</code>	센서 ON/OFF와 저장 여부 설정
GET	<code>/crew</code>	현재 선택 선원 조회
POST	<code>/crew</code>	현재 선택 선원 설정
GET	<code>/crew/focus</code>	집중 관리 선원 목록 조회
POST	<code>/crew/focus</code>	집중 관리 선원 목록 저장
POST	<code>/trauma/start</code>	Jetson PyQt5 외상 화면 시작
POST	<code>/trauma/capture</code>	Jetson PyQt5 촬영/분석 시작
GET	<code>/trauma/frame.jpg</code>	Jetson 카메라 프레임 프록시
GET	<code>/trauma/result</code>	Jetson 외상 분석 결과 조회
POST	<code>/trauma/guide</code>	Jetson 응급처치 가이드 전환

Method	Endpoint	역할
POST	/trauma/stop	Jetson 카메라 송출 중단

Jetson PyQt5 Control API

Jetson의 `main.py` 는 내부 HTTP 서버를 실행해 웹과 Pi가 PyQt5 화면을 제어할 수 있게 한다.

Method	Endpoint	역할
GET	/health	PyQt5 제어 API 상태 확인
GET	/trauma/frame.jpg	최신 카메라 JPEG 프레임 반환
GET	/trauma/result	외상 분석 상태/결과 반환
POST	/trauma/start	외상 촬영 화면 표시
POST	/trauma/capture	촬영/분석 시작
POST	/trauma/guide	외상 가이드 화면 전환
POST	/trauma/stop	카메라 송출 및 분석 상태 중단

PyQt5 화면 구성

1. MonitorScreen

현재 선택된 환자의 바이탈을 표시한다. 환자가 선택되지 않은 상태에서는 센서값을 표시하지 않고 대기 상태로 둔다.

표시 항목은 다음과 같다.

- 심박수
- 산소포화도
- 호흡수
- 혈압
- 체온
- 환자 정보
- 센서 연결 상태

2. CrewScreen

웹 선원 관리와 동일한 선원 목록을 표시한다. 원격 `tb_crew` 조회에 성공하면 DB 선원 데이터를 사용하고, 실패하면 로컬 fallback 데이터를 사용한다.

- 전체 선원
- 응급 환자
- 항해부

- 기관부
- 조리/지원
- 환자전환/관리 중 버튼

PyQt5에서 환자전환을 누르면 해당 선원은 집중 관리 상태로 Pi 서버에 저장되고 웹에도 반영된다.

3. ControlScreen

센서 ON/OFF와 DB 저장 게이트를 제어한다.

- 산소포화도 / 심박수 / 호흡수
- 체온 측정
- 외상 촬영
- 모니터링 시작

센서가 꺼진 상태에서는 이전 수기 입력값이나 이전 센서값이 계속 DB에 저장되지 않도록 `/recording` 과 `/manual/clear` 를 사용한다.

4. TraumaScanScreen

Jetson 카메라를 사용해 외상 촬영 UI를 표시한다. 웹 대시보드에는 이 화면의 최신 JPEG 프레임이 전달된다.

- 촬영 시작
- 스캐너 애니메이션
- 분석 진행률
- 분석 완료 모달
- 다시 촬영하기
- 응급처치가이드
- 진단 모드 종료

5. TraumaGuideScreen

외상 분석 결과에 따라 찰과상, 타박상, 화상, 절상, 열상, 자창 등 응급처치 가이드를 표시한다. 웹에서 AI 분석 시작을 누르면 PyQt5도 동일 가이드로 전환된다.

DB 저장 게이트

센서 서버는 다음 조건을 만족할 때만 DB에 저장한다.

1. PyQt5에서 센서 또는 체온 측정을 켜 상태
2. 유효한 센서값 또는 수기 입력값이 존재
3. 마지막 유효 입력 후 idle timeout 이내
4. 선택된 선원 `crew_id` 가 존재

이 구조는 센서를 꺼둔 상태에서 이전 값이 계속 원격 DB로 전송되는 문제를 막는다.

집중 관리 상태 저장 파일

Raspberry Pi는 집중 관리 상태를 다음 파일에 저장한다.

```
/home/pi/mdts_focused_crew_state.json
```

예시 구조는 다음과 같다.

```
{
  "focused_crew_ids": [3, 4, 6],
  "updated_at": "2026-05-13 16:20:00",
  "source": "web-crew-management"
}
```

이 파일은 원격 DB를 수정하지 않고 웹과 PyQt5 상태를 맞추기 위한 시연용 상태 저장소다.

실행 방법

Raspberry Pi 센서 서버

```
cd /home/pi
python3 -u /home/pi/sensor_server_rpi.py
```

백그라운드 실행은 다음과 같다.

```
cd /home/pi
nohup python3 -u /home/pi/sensor_server_rpi.py > /home/pi/sensor.log 2>&1 < /dev/null
&
```

Jetson Nano PyQt5

```
cd /home/jetson
DISPLAY=:0 QT_QPA_PLATFORM=xcb python3 -u /home/jetson/main.py
```

백그라운드 실행은 다음과 같다.

```
cd /home/jetson
DISPLAY=:0 QT_QPA_PLATFORM=xcb nohup python3 -u /home/jetson/main.py >
/home/jetson/pyqt5.log 2>&1 < /dev/null &
```

배포 절차

Raspberry Pi 배포

```
scp sensor_server_rpi.py pi@<raspberrypi>:/home/pi/sensor_server_rpi.py
ssh pi@<raspberrypi>
pkill -9 -f '[s]ensor_server_rpi.py' || true
cd /home/pi
nohup python3 -u /home/pi/sensor_server_rpi.py > /home/pi/sensor.log 2>&1 < /dev/null &
```

Jetson Nano 배포

```
scp main.py jetson@<jetson-nano>:/home/jetson/main.py
ssh jetson@<jetson-nano>
pkill -9 -f '[m]ain.py' || true
cd /home/jetson
DISPLAY=:0 QT_QPA_PLATFORM=xcb nohup python3 -u /home/jetson/main.py >
/home/jetson/pyqt5.log 2>&1 < /dev/null &
```

직접 LAN 연결 상태에서는 Windows에서 Jetson에 직접 접근하지 못할 수 있으므로 Raspberry Pi를 SSH jump 경로로 사용한다.

센서 하드웨어

센서	기본 I2C 주소	역할
MAX30102	0x57	심박수, 산소포화도
MLX90614	0x5A	비접촉 체온
ADS1115	0x48	아날로그 혈압/호흡 입력

메모리 최적화 기준

Jetson Nano는 메모리가 제한적이므로 다음 기준을 지킨다.

- PyQt5 카메라 프레임은 최신 JPEG만 유지한다.
- 외상 촬영 종료 시 카메라 타이머, 스캔 타이머, 최신 프레임을 모두 제거한다.
- 웹 메인으로 돌아갈 때 PyQt5 촬영 상태를 중단한다.
- Ollama 모델은 필요 시 keep_alive를 짧게 유지한다.
- 불필요한 프로세스가 남아 있으면 `pkill` 로 정리한다.

트러블슈팅

증상	원인	조치
PyQt5 화면이 안 뜸	DISPLAY 또는 X11 문제	DISPLAY=:0 , HDMI 화면, X server 상태 확인

증상	원인	조치
웹에서 카메라가 계속 커짐	PyQt5 촬영 상태 미중단	<code>/trauma/stop</code> 호출, PyQt5 재시작 확인
센서값이 0으로 표시됨	손가락 미감지 또는 센서 OFF	센서 접촉, <code>/recording</code> , I2C 상태 확인
체온 센서를 껐는데 값이 저장됨	manual 값 캐시	<code>/manual/clear</code> 와 temp sensor gate 확인
PyQt5와 웹 집중 관리 상태가 다름	Pi <code>/crew/focus</code> 상태 불일치	<code>/home/pi/mdts_focused_crew_state.json</code> 과 API 확인
원격 DB에 저장되지 않음	선택 선원 없음	PyQt5에서 선원 선택 후 측정
I2C 오류 발생	버스 충돌 또는 장치 미감지	기존 센서 프로세스 종료, I2C 모듈 재로드
Jetson 접속 불가	직접 LAN/Wi-Fi 경로 차이	Pi jump 경로 또는 현재 네트워크 모드 확인

운영 체크리스트

- Raspberry Pi 센서 서버 실행 확인
- Jetson PyQt5 화면 실행 확인
- 웹에서 `/api/sensor/live` 조회 확인
- PyQt5에서 선원 선택 후 웹 메인 대상자 변경 확인
- 웹에서 환자로 전환 후 PyQt5 `관리 중` 표시 확인
- PyQt5에서 환자전환 후 웹 `집중 관리 중` 표시 확인
- 센서 OFF 시 DB 저장 중단 확인
- 외상 촬영 시작/분석/중단 확인
- 응급처치 데이터 전송 후 환자 차트 저장 확인

보안 및 운영 주의

이 모듈은 장비 제어 코드와 내부망 주소를 포함한다. 저장소는 private으로 유지하고, 실제 운영 전에는 다음을 진행한다.

- SSH/DB 접속정보를 환경 변수 또는 장비별 설정 파일로 분리
- Flask API에 내부망 접근 제한 또는 인증 추가
- 의료정보/개인정보 로그 최소화
- 장애 발생 시 USB CSV 백업과 로컬 MariaDB를 우선 보존
- 원격 DB 장애 시에도 로컬 저장이 지속되는지 확인

개발 원칙

- PyQt5와 웹의 상태가 달라질 수 있는 값은 Pi 서버 또는 DB를 기준으로 공유한다.
- 센서값 저장은 사용자의 명시적 센서 ON 상태를 기준으로 한다.

- 외상 촬영 UI는 웹과 PyQt5가 동일 이벤트 흐름을 따라야 한다.
- 의료 답변이나 응급처치 가이드는 DB/센서 상태와 분리하지 않는다.
- 장비 메모리 부족이 보이면 카메라, Ollama, 중복 Python 프로세스를 우선 점검한다.

문서 기준일

- 문서 기준일: 2026-05-13
- 대상 저장소: `Capernaum-user/mdts-maritime-medic-integrated-demo`
- 대상 모듈: `device_sensor_pyqt5`